Lecture 12: Defining sameness for complex data, part 2

Using double-dispatch to test for sameness

Video Lessons

- Lesson 1 - Sameness using double dispatch

Motivation

In Lecture 11: Defining sameness for complex data, part 1 we took up the question, "How can we test when two values are 'the same'?" We introduced two new language mechanisms: *casting* to try to convince Java to treat an identifier as if it were declared at a more specific type, and `instanceof` to check whether a given value actually is of a particular type. But our implementations were brittle and suffered from clumsy helper methods that were not useful outside this limited context, and still potentially could throw exceptions and crash. Plus, our design sense should be warning us: any time we feel tempted to try to do a type-test in Java, we ought to find a better way...preferably using dynamic dispatch.

Recall our desired properties for a sameness test:

- **Reflexivity:** every object should be the same as itself.
- **Symmetry:** if object `x` is the same as object `y`, then `y` is the same as `x`.
- **Transitivity:** if two objects are both the same as a third object, then they are the same as each other.
- **Totality:** we can compare any two objects of the same type, and obtain a correct answer.

Let's also bring back our tests from last time:

```
// In test method in an Examples class
Circle c1 = new Circle(3, 4, 5);
```

```
Circle c2 = new Circle(4, 5, 6);
Circle c3 = new Circle(3, 4, 5);
Rect r1 = new Rect(3, 4, 5, 5);
Rect r2 = new Rect(4, 5, 6, 7);
Rect r3 = new Rect(3, 4, 5, 5);
Square s1 = new Square(3, 4, 5);
Square s2 = new Square(4, 5, 6);
Square s3 = new Square(3, 4, 5);

t.checkExpect(c1.sameCircle(c2), false)
t.checkExpect(c2.sameCircle(c1), false)
t.checkExpect(c1.sameCircle(c3), true)
t.checkExpect(c3.sameCircle(c1), true)

t.checkExpect(r1.sameRect(r2), false)
t.checkExpect(r2.sameRect(r1), false)
t.checkExpect(r1.sameRect(r3), true)
t.checkExpect(r3.sameRect(r1), true)

t.checkExpect(s1.sameShape(s2), false)
t.checkExpect(s2.sameShape(s1), false)
t.checkExpect(s1.sameShape(s3), true)
t.checkExpect(s3.sameShape(s1), true)

// Comparing a Square with a Rect of a different size
t.checkExpect(s1.sameShape(r2), false)
t.checkExpect(r2.sameShape(s1), false)
// Comparing a Square with a Rect of the same size
t.checkExpect(s1.sameShape(r1), false)
t.checkExpect(r1.sameShape(s1), false)
```

To design a better mechanism for checking sameness...we have actually already written nearly all the helpers we need. We just need to use them more cleverly!

12.1 Sameness for unions: Successful attempt using double-dispatch

Let's retrace our steps to see what went wrong when trying to define `sameShape` for the `Circle` class:

```
// In Circle:
public boolean sameShape(IShape that) {
  /*
```

```
    * Fields:
    * this.x, this.y, this.radius
    *
    * Methods:
    * this.sameShape(IShape) -> boolean
    * this.sameCircle(Circle) -> boolean
    *
    * Methods on fields:
    *
    * Fields of parameters:
    *
    * Methods on parameters:
    * that.sameShape(IShape) -> boolean
    */
    ???
  }
```

We had nothing in our template that we could use: we didn't have enough information about `that` to determine what sort of shape it was.

Suppose we had the following idea: the `sameCircle`, `sameRect` and `sameSquare` methods were at least somewhat useful, so let's add those methods to the `IShape` interface.

```
interface IShape {
  boolean sameShape(IShape that);
  boolean sameCircle(Circle that);
  boolean sameRect(Rect that);
  boolean sameSquare(Square that);
}
```

> ***Do Now!***
>
> Now the `Circle` class needs implementations for `sameRect` and `sameSquare`, the `Rect` class is missing `sameCircle` and `sameSquare`, and `Square` is missing `sameCircle` and `sameRect`. Implement them — they should be very short!

Now let's look at how our template has changed:

```
// In Circle:
public boolean sameShape(IShape that) {
  /*
   * Fields:
   * this.x, this.y, this.radius
```

```
    *
    * Methods:
    * this.sameShape(IShape) -> boolean
    * this.sameCircle(Circle) -> boolean
    * this.sameRect(Rect) -> boolean
    * this.sameSquare(Square) -> boolean
    *
    * Methods on fields:
    *
    * Fields of parameters:
    *
    * Methods on parameters:
    * that.sameShape(IShape) -> boolean
    * that.sameCircle(Circle) -> boolean
    * that.sameRect(Rect) -> boolean
    * that.sameSquare(Square) -> boolean
    */
    ???
}
```

We have several more methods to choose from; perhaps we can use one of them to finish our method.

> ***Do Now!***
>
> Looking at the types for `this` and `that`, which method in the template is most likely to be useful?

> We'd be going around in circles...

We could try invoking `that.sameShape(this)`, but that doesn't really make any progress. (In fact, if `that` is also a `Circle`, we'd end up stuck in an infinite recursion, with no base case.) We know that `that` is some `IShape`. And we know—because we're in the `Circle` class—that *`this` is a `Circle`*, and not just another `IShape`. Which means we can invoke `that.sameCircle(...)` using `this` as the argument:

```
// In Circle:
public boolean sameShape(IShape that) {
  return that.sameCircle(this);
}
```

And we're done! Let's trace through a few examples: if we test `c1.sameShape(c3)`:

- `c1` is a `Circle`, so we invoke the `sameShape` method defined in `Circle`, inside of which `this` is `c1` and `that` is `c3`.
- This method immediately invokes `sameCircle` on `that`, and passes it `this`. Because `that` is `c3`, and `c3` is a `Circle`, we invoke the `sameCircle` method defined in the `Circle` class, inside of which `this` will be `c3` and `that` will be `c1` — they've swapped roles.
- Inside `sameCircle`, `this` is now `c3` and `that` is now `c1`, and we can easily compare their fields. The fields are all equal, so we return `true`.

If we try a similar test with `c1.sameShape(c2)`:

- `c1` is a `Circle`, so we invoke the `sameShape` method defined in `Circle`, inside of which `this` is `c1` and `that` is `c2`.
- This method immediately invokes `sameCircle` on `that`, and passes it `this`. Because `that` is `c2`, and `c2` is a `Circle`, we invoke the `sameCircle` method defined in the `Circle` class, inside of which `this` will be `c2` and `that` will be `c1` — they've swapped roles.
- Inside `sameCircle`, `this` is now `c2` and `that` is now `c1`, and we can easily compare their fields. The fields are not equal, so we return `false`.

Perfect — we can compare `Circle`s correctly.

Let's try something trickier: let's test `c1.sameShape(r1)`:

- `c1` is a `Circle`, so we invoke the `sameShape` method defined in `Circle`, inside of which `this` is `c1` and `that` is `r1`.
- This method immediately invokes `sameCircle` on `that`, and passes it `this`. Because `that` is `r1`, and `r1` is a `Rect`, we invoke the `sameCircle` method defined in the `Rect` class, inside of which `this` will be `r1` and `that` will be `c1` — they've swapped roles.
- ...

Oops — we haven't implemented `sameCircle` for the `Rect` class yet. But it's easy: read through the purpose statement in your head, "when is this `Rect` the same `Circle` as that `Circle`?" Answer: never!

```
// In Rect
public boolean sameCircle(Circle that) { return false; }
```

In fact, we can implement all the "mismatched" methods this way:

```
// In Circle
public boolean sameRect(Rect that)     { return false; }
public boolean sameSquare(Square that) { return false; }

// In Rect
public boolean sameCircle(Circle that) { return false; }
public boolean sameSquare(Square that) { return false; }

// In Square
public boolean sameCircle(Circle that) { return false; }
public boolean sameRect(Rect that)     { return false; }
```

With these methods defined, we can finish the trace above:

- The `sameCircle` method in the `Rect` class always returns `false`: `Circle`s and `Rect`s are never the same.

Now let's try the two trickiest examples: comparing a `Square` with a `Rect` and symmetrically a `Rect` with a `Square`:

> ***Do Now!***
>
> Do this yourself, following the pattern of the step-by-step walkthroughs above.

To compare `s1.sameShape(r1)`:

- `s1` is a `Square`, so we invoke the `sameShape` method defined in `Square`, inside of which `this` is `s1` and `that` is `r1`.
- This method immediately invokes `sameSquare` on `that`, and passes it `this`. Because `that` is `r1`, and `r1` is a `Rect`, we invoke the `sameSquare` method defined in the `Rect` class, inside of which `this` will be `r1` and `that` will be `s1` — they've swapped roles.
- The `sameSquare` method in the `Rect` class always returns `false`: `Square`s and `Rect`s are never the same.

Conversely, to compare `r1.sameShape(s1)`:

- `r1` is a `Rect`, so we invoke the `sameShape` method defined in `Rect`, inside of which `this` is `r1` and `that` is `s1`.
- This method immediately invokes `sameRect` on `that`, and passes it `this`. Because `that` is `s1`, and `s1` is a `Square`, we invoke the `sameRect` method defined in the `Square` class, inside of which `this` will be `s1` and `that` will be `r1` — they've swapped

roles.

- The `sameRect` method in the `Square` class always returns `false`: `Rect`s and `Square`s are never the same.

Perfect!

12.2 Summarizing the double-dispatch pattern for sameness testing

Suppose we have an interface `IFoo`, and classes `X`, `Y` and `Z` that implement this interface. In the interface, we define a method `sameFoo` that takes an `IFoo`, and we define one method for each class that implements the interface:

```
interface IFoo {
  boolean sameFoo(IFoo that);

  boolean sameX(X that);
  boolean sameY(Y that);
  boolean sameZ(Z that);
}
```

In each class, the interesting behavior is in the `sameFoo` method and the method with the same name as the class:

```
// In X:
public boolean sameFoo(IFoo that) { return that.sameX(this); }
public boolean sameX(X that) { ... compares two X values ... }
```

All the other methods just return false:

```
// In X:
public boolean sameY(Y that) { return false; }
public boolean sameZ(Z that) { return false; }
```

How does this all work? The key observation is that when we want to compare two `IFoo` values `f1` and `f2` for sameness, we need to determine which kind of `IFoo` `f1` is, *and also* which kind of `IFoo` `f2` is. We get the answer to the first part automatically: by invoking a method on `f1`, dynamic dispatch will automatically route us to the appropriate method for the actual type of `f1`. Then, instead of trying to cobble together some form of type testing to figure out the answer for `f2`, we just invoke a method on it, using dispatch again to figure out the answer there. The trick lies in using our helper methods cleverly: by having `sameFoo` invoke `sameX` (in the example above), we

are essentially answering the question "is `f1` the same `IFoo` as `f2`?" by rebounding the question: "well we've figured out that `f1` is an `X`, so is `f2` the same `X` as `f1`?"

This pattern, of using *two* method invocations to get the benefits of dynamic dispatch twice, is called *double dispatch*, and it is a very common and very elegant pattern.

Exercise

Look back at Lecture 7: Accumulator methods, continued. Where did we use the double-dispatch pattern to answer a similar kind of question, that had to distinguish between different variants of a union data type?

Do Now!

Confirm that all of the properties we want out of a sameness operation are satisfied by our implementation. Notice in particular that totality is easy to confirm: there are no exceptions possible in this code!

Exercise

Implement a sameness operator for two `IAT` ancestry trees.

12.3 Cleaning up the code

You may have noticed that there are a *lot* of methods in the pattern above that always return `false`. (For *n* variants, there are *n * n* helper methods, of which *n * (n - 1)* of them should always return `false`.) In fact, all of the `same`*`Class`* methods always return `false`, except in one case in each class. This sort of pattern, where the behavior is always the same except for a few cases where it differs, calls for the abstraction recipe for methods: we create an abstract class, put the default versions of these methods in there once and for all, then override them as needed in each subclass:

```
abstract class AFoo implements IFoo {
  public boolean sameX(X that) { return false; }
  public boolean sameY(Y that) { return false; }
  public boolean sameZ(Z that) { return false; }
}
class X extends AFoo {
  public boolean sameFoo(IFoo that) { return that.sameX(this); }
```

```
  public boolean sameX(X that) { ... compares two X values ... }
}
class Y extends AFoo {
  public boolean sameFoo(IFoo that) { return that.sameY(this); }
  public boolean sameY(Y that) { ... compares two Y values ... }
}
class Z extends AFoo {
  public boolean sameFoo(IFoo that) { return that.sameZ(this); }
  public boolean sameZ(Z that) { ... compares two Z values ... }
}
```

Exercise

Implement an `AShape` class, and abstract all the default cases you can. Be careful with the `Square` class!

12.4 Adding more variants

We also defined a `Combo` class as a kind of `IShape`. Let's add it now, and implement sameness for it:

```
interface IShape {
  boolean sameShape(IShape that);
  boolean sameCircle(Circle that);
  boolean sameRect(Rect that);
  boolean sameSquare(Square that);
  // NEW:
  boolean sameCombo(Combo that);
}
// in AShape:
public boolean sameCombo(Combo that) { return false; }
```

Notice that because of inheritance from `AShape`, we don't need to change `Circle`, `Rect` or `Square` at all.

```
// in Combo:
public boolean sameShape(IShape that) { return that.sameCombo(this); }
public boolean sameCombo(Combo that) {
  ...
}
```

To compare two `Combo`s for sameness, we need to compare their `left` sides for

sameness and their `right` sides for sameness. Both `left` and `right` are `IShape`s; conveniently, we have a method to compare two `IShape`s for sameness!

```
// In Combo
public boolean sameCombo(Combo that) {
  return this.left.sameShape(that.left) &&
         this.right.sameShape(that.right);
}
```

Trace through a few examples to convince yourself that this method—using double dispatch in recursive calls!—works properly.